Guía Maestra de Git y GitHub: De los Fundamentos a la Excelencia Profesional

Documento Pedagógico de Estudio y Preparación de Clase / Exposición
Área: Ingeniería de Software y Sistemas — Control de Versiones y Trabajo Colaborativo
Dirigido a: Estudiantes de Ingeniería de Sistemas — Universidad de la Amazonia
Objetivo: Dominar el modelo mental de Git, la plataforma GitHub, los comandos esenciales y comprender por qué constituyen el pilar fundamental del perfil profesional de un ingeniero.

📑 Tabla de Contenidos

1. **La Problemática Real: ¿Por qué necesitamos Control de Versiones?**
2. **Git vs. GitHub: La Distinción Fundamental (Pregunta de Examen)**
3. **El Modelo Mental de Git: Las 3 + 1 Áreas del Ciclo de Vida**
4. **Manual Táctico de Comandos Git (Paso a Paso sin Obviar Nada)**
5. **4.1 Configuración Inicial y Credenciales**
6. **4.2 Creación e Inspección**
7. **4.3 El Ciclo de Guardado Local (Add, Commit, Status, Diff)**
8. **4.4 Ramas (Branches), Navegación y Fusiones (Merge vs Rebase)**
9. **4.5 Sincronización Remota (Remote, Fetch, Pull, Push)**
10. **4.6 Herramientas de Rescate y Auditoría (Stash, Log, Blame, Reset)**
11. **GitHub en la Práctica: Ecosistema de Ingeniería y Colaboración**
12. **5.1 Anatomía de un Repositorio Profesional**
13. **5.2 Flujo de Pull Requests (PR) y Revisiones de Código**
14. **5.3 Gestión Ágil: Issues, Milestones y Projects (Kanban)**
15. **5.4 Automatización Moderna: GitHub Actions (CI/CD)**
16. **¿Por qué Git y GitHub son Cruciales para la Hoja de Vida y la Carrera?**
17. **Estructura y Guion Pedagógico para la Exposición**
18. **Enlaces y Recursos Curados (Videos, Libros y Simuladores Interactivos)**

1. La Problemática Real: ¿Por qué necesitamos Control de Versiones?

En el desarrollo de software y en la ingeniería moderna, un proyecto evoluciona constantemente. Antes de los sistemas de control de versiones, la gestión de cambios en archivos de código dependía de métodos rudimentarios que generaban graves pérdidas económicas y técnicas.

El Caos del Enfoque Tradicional

- Duplicación infinita de carpetas: `text SistemaTucanGo_v1/ SistemaTucanGo_v2_modificado/ SistemaTucanGo_v2_final/ SistemaTucanGo_v2_final_REAL/ SistemaTucanGo_v2_final_este_si_es_definitivo.zip`
- **Imposibilidad de trabajo concurrente:** Si dos desarrolladores modificaban el mismo archivo `.java` al mismo tiempo, el último en guardar sobrescribía y destruía el código del primero.
- **Cero trazabilidad forense:** Imposible saber quién introdujo un bug crítico, en qué fecha exacta se alteró una función o por qué se tomó una decisión arquitectónica.

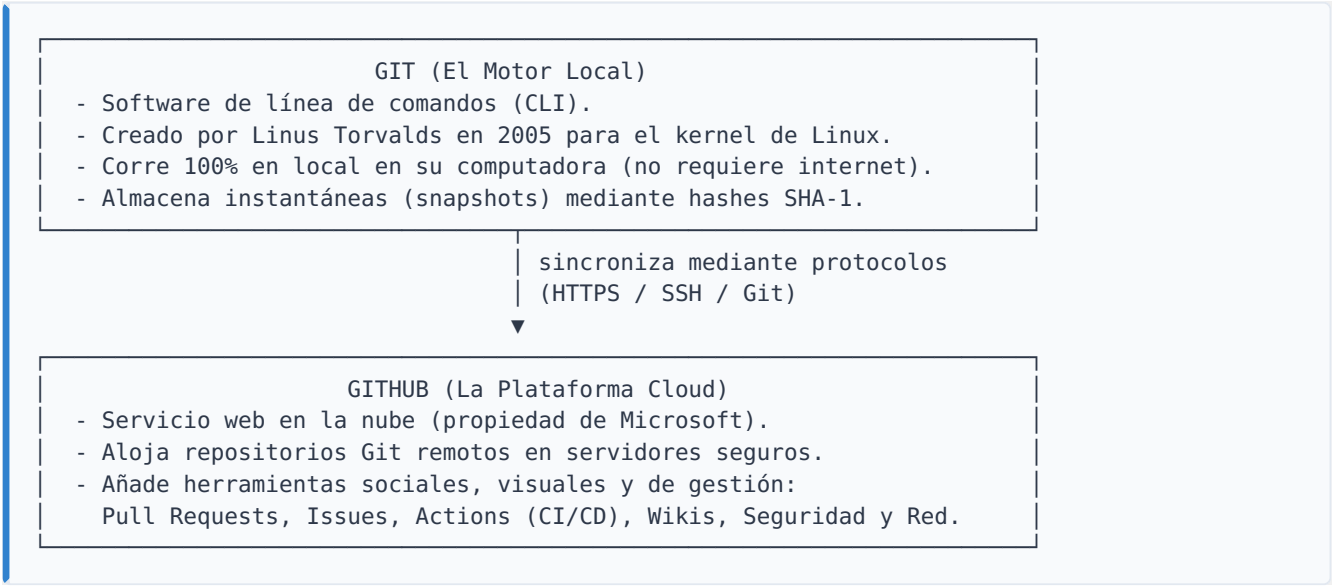
La Solución: Sistemas de Control de Versiones Distribuidos (DVCS)

Un Sistema de Control de Versiones es un software que registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo, permitiendo: 1. Revertir archivos o proyectos enteros a estados previos exactos. 2. Comparar cambios línea por línea a través del tiempo. 3. Trabajar en ramas paralelas de desarrollo sin interferir con el código estable en producción.

---

2. Git vs. GitHub: La Distinción Fundamental (Pregunta de Examen)

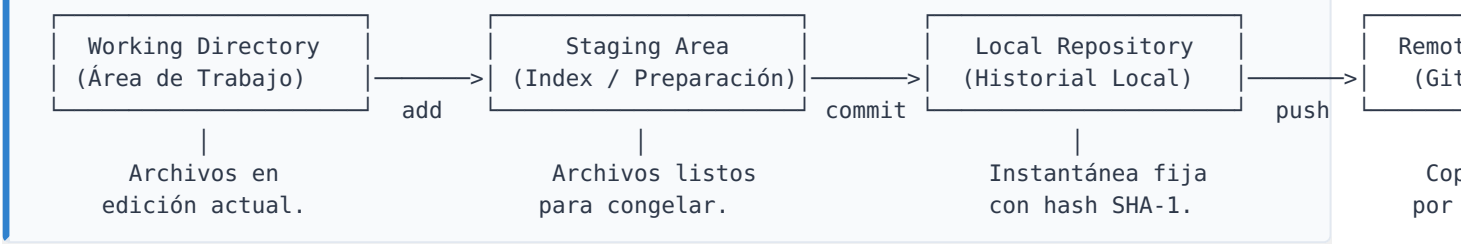
Una de las confusiones más comunes en estudiantes novatos —y que los docentes de ingeniería evalúan con rigor— es asumir que Git y GitHub son lo mismo.



Criterio	Git	GitHub
Naturaleza	Herramienta / Software instalable localmente.	Plataforma web / Servicio SaaS en la nube.
Requiere Internet	**No**. Todo el historial vive en la carpeta `.git/`.	**Sí**, para interactuar con la nube y sincronizar.
Función Principal	Rastrear cambios, crear ramas, commits y merges.	Alojar repositorios remotos y facilitar colaboración.
Competidores	Mercurial, SVN, Bazaar.	GitLab, Bitbucket, Azure DevOps, Gitea.

3. El Modelo Mental de Git: Las 3 + 1 Áreas del Ciclo de Vida

Para entender Git no hay que memorizar comandos; hay que comprender en qué estado y lugar físico/lógico se encuentran los archivos en cada momento.



1. **Working Directory (Directorio de Trabajo):** Es la carpeta visible donde usted abre su editor (VS Code, NetBeans) y modifica código. Los cambios aquí aún no están protegidos por Git.
2. **Staging Area (Área de Preparación / Index):** Es un archivo intermedio dentro de `.git/` que almacena la información sobre los cambios que se incluirán en la próxima "foto" o versión. Permite construir commits limpios y atómicos.
3. **Local Repository (Repositorio Local):** Es la base de datos interna (`.git/objects/`) donde Git guarda permanentemente las instantáneas (*commits*) con sus metadatos (autor, fecha, mensaje, hash SHA-1).
4. **Remote Repository (Repositorio Remoto):** El servidor en la nube (GitHub) que actúa como punto de encuentro para que múltiples ingenieros sincronicen sus repositorios locales.

4. Manual Táctico de Comandos Git (Paso a Paso sin Obviar Nada)

4.1 Configuración Inicial y Credenciales

Antes de hacer cualquier commit, Git debe saber la identidad del autor:

```
# Configurar el nombre del autor
git config --global user.name "Gian Marco Castañeda"

# Configurar el correo (debe coincidir con el de GitHub)
git config --global user.email "marco@uniamazonia.edu.co"

# Configurar la rama principal por defecto como 'main'
git config --global init.defaultBranch main

# Verificar toda la configuración activa
git config --list
```

4.2 Creación e Inspección

```
# Iniciar un nuevo repositorio local en la carpeta actual (crea la carpeta oculta .git)
git init

# Clonar un repositorio remoto existente desde GitHub hacia la máquina local
git clone https://github.com/Stellel-One/TucanGo.git

# Inspeccionar el estado del árbol de trabajo (archivos modificados, staged o sin seguimiento)
git status
```

4.3 El Ciclo de Guardado Local (Add, Commit, Status, Diff)

```
# Mover un archivo específico del Working Directory al Staging Area
git add src/co/edu/uniamazonia/logica2/modelo/Persona.java

# Mover TODOS los archivos modificados y nuevos al Staging Area
git add .

# Ver qué líneas exactas cambiaron en el Working Directory antes de hacer add
git diff

# Ver qué cambios están en el Staging Area listos para ser commiteados
git diff --staged

# Congelar los cambios preparados en un commit con un mensaje descriptivo
git commit -m "feat(modelo): agrega clase abstracta Persona con atributos base"
```

> 📌 **Regla de Oro del Commit:** Un commit debe ser **atómico** (hace una sola cosa bien hecha) y su mensaje debe explicar *qué* se hizo y *por qué*, no solo "cambios".

4.4 Ramas (Branches), Navegación y Fusiones (Merge vs Rebase)

Las ramas permiten bifurcar la línea de desarrollo para crear nuevas funcionalidades sin romper el código estable.

```
# Listar todas las ramas locales (* indica en cuál estás parado)
git branch

# Crear una nueva rama para una característica específica
git branch feat/sistema-pagos

# Cambiarse a la rama creada
git switch feat/sistema-pagos
# (En versiones antiguas de Git: git checkout feat/sistema-pagos)

# Atajo: Crear la rama y cambiarse a ella en un solo paso
git switch -c feat/sistema-pagos

# Fusionar la rama 'feat/sistema-pagos' dentro de la rama actual (por ej. dentro de main)
git merge feat/sistema-pagos

# Eliminar una rama local que ya fue integrada y no se necesita más
git branch -d feat/sistema-pagos
```

Merge vs. Rebase:

- `git merge`**:** Une dos historias preservando la bifurcación exacta y creando un *commit de merge*. Es transparente y no altera el pasado.
- `git rebase`**:** Traslada la base de su rama al último commit de la rama destino, generando un historial lineal y limpio (debe usarse con precaución en ramas compartidas).

4.5 Sincronización Remota (Remote, Fetch, Pull, Push)

```
# Ver los servidores remotos configurados (normalmente llamado 'origin')
git remote -v

# Vincular un repositorio local a uno remoto en GitHub
git remote add origin https://github.com/Stellel-One/TucanGo.git

# Subir la rama local 'main' al servidor remoto 'origin' por primera vez (-u configura el upstream)
git push -u origin main

# Subidas posteriores (una vez configurado el upstream)
git push

# Descargar las novedades del servidor remoto SIN mezclarlas con tu código local todavía
git fetch origin

# Descargar las novedades e integrarlas inmediatamente en tu rama local activa (fetch + merge)
git pull origin main
```

4.6 Herramientas de Rescate y Auditoría (Stash, Log, Blame, Reset)

```
# Guardar temporalmente cambios en un "cajón secreto" para limpiar el directorio sin commitear
git stash

# Recuperar los cambios guardados en el stash
git stash pop

# Ver el historial de commits gráfico y compacto
git log --oneline --graph --all

# Inspeccionar línea por línea quién modificó un archivo y en qué commit
git blame src/co/edu/uniamazonia/logica2/modelo/Motorista.java

# Deshacer cambios en un archivo no preparado (restaurarlo al último commit)
git restore src/co/edu/uniamazonia/logica2/Main.java

# Sacar un archivo del Staging Area sin perder lo que escribiste
git restore --staged src/co/edu/uniamazonia/logica2/Main.java
```

---

5. GitHub en la Práctica: Ecosistema de Ingeniería y Colaboración

GitHub no es solo un disco duro en la nube; es una plataforma completa de **Ingeniería y Entrega Continua**.

5.1 Anatomía de un Repositorio Profesional

Un repositorio de ingeniería de alto nivel contiene: 1. `README.md`**:** La portada del proyecto. Explica qué hace, cómo se instala, cómo se compila y quiénes lo crearon. 2. `.gitignore`**:** Lista de archivos que Git DEBE ignorar (archivos binarios `.class`, carpetas de compilación `bin/`, `build/`, variables secretas `.env`, archivos del sistema operativo `.DS_Store`). 3. `LICENSE`**:** Términos legales de uso (MIT, GPLv3, Apache 2.0, Académica). 4. **Carpeta** `.github/`**:** Configuración de flujos automáticos (*workflows*).

5.2 Flujo de Pull Requests (PR) y Revisiones de Código

El **Pull Request (PR)** es el mecanismo estándar de la industria para integrar código: 1. Un desarrollador trabaja en su rama (`feat/login-estudiante`). 2. Sube su rama a GitHub (`git push origin feat/login-estudiante`). 3. Abre un **Pull Request** solicitando fusionar sus cambios hacia `main`. 4. El equipo (o el líder técnico) revisa el *diff* línea por línea, comenta dudas o exige correcciones. 5. Una vez aprobado y pasados los tests automáticos, se hace el **Merge**.

5.3 Gestión Ágil: Issues, Milestones y Projects (Kanban)

- **Issues:** Reporte estructurado de errores (*bugs*), solicitudes de nuevas funcionalidades (*features*) o tareas pendientes.
- **Milestones (Hitos):** Fechas límite asociadas a un conjunto de entregables (ej. "Entrega Parcial 1").
- **GitHub Projects:** Tableros Kanban interactivos (*To Do*, *In Progress*, *Done*) directamente vinculados a los commits y PRs del repositorio.

5.4 Automatización Moderna: GitHub Actions (CI/CD)

GitHub Actions permite ejecutar código automáticamente en servidores de GitHub ante cualquier evento (`push` o `pull_request`): * Compilar el proyecto en Java 21. * Ejecutar toda la batería de pruebas unitarias (JUnit). * Rechazar el Pull Request si alguna prueba falla o el código no compila.

---

6. ¿Por qué Git y GitHub son Cruciales para la Hoja de Vida y la Carrera?

Para un estudiante o graduado de Ingeniería de Sistemas, **GitHub es el currículum del siglo XXI**.

```
┌──────────────────────────────────────────────────────────┐
│                  EL PARADIGMA DE LA CONTRATACIÓN         │
│                                                          │
│  ❌ Currículum Tradicional en Papel:                     │
│     "Dominio avanzado de Java, POO, patrones y trabajo en equipo."     │
│     (Cualquiera puede escribirlo; no demuestra nada).    │
│                                                          │
│  ✅ Perfil de GitHub con Repositorios Reales:            │
│     • Enlace directo a código fuente verificable.        │
│     • Historial de commits ordenados y constantes.       │
│     • Pull Requests con argumentos técnicos y revisiones.│
│     • Arquitecturas reales y pruebas unitarias pasando.  │
└──────────────────────────────────────────────────────────┘
```

4 Razones Fundamentales para el Ingeniero:

1. **Evidencia Empírica de Competencia (*Proof of Work*):** Las empresas de tecnología contratan basándose en lo que usted puede demostrar. Un perfil con repositorios bien estructurados demuestra disciplina, estándares y orden.
2. **Cultura de Trabajo en Equipo Real:** El código en las empresas nunca lo escribe una sola persona. Saber resolver conflictos de merge, redactar PRs claros y aceptar revisiones de código es una habilidad blanda/dura más valorada que la pura sintaxis.
3. **Respaldo e Inmunidad ante Desastres:** Los discos duros fallan, las laptops se dañan y los archivos locales se corrompen. Un proyecto en GitHub con control de versiones distribuido es indestructible; se recupera en 10 segundos con un `git clone`.
4. **Acceso al Ecosistema Open Source:** Permite colaborar con herramientas usadas a nivel mundial (Linux, React, Spring Boot) y ganar visibilidad ante reclutadores internacionales.

---

7. Estructura y Guion Pedagógico para la Exposición

Si debe liderar la clase o exposición ante el docente, esta es la secuencia recomendada de 15 a 20 minutos:

```
[MINUTO 0-3]  ———> INTRODUCCIÓN: La pesadilla del archivo "final_v2_este_si.zip".
[MINUTO 3-6]  ———> TEORÍA CLAVE: Git (motor local) vs GitHub (plataforma colaborativa).
[MINUTO 6-10] ———> EL MODELO MENTAL: Explicar las 3 áreas (Working Dir -> Staging -> Repo).
[MINUTO 10-15]———> DEMO EN VIVO: Mostrar un caso real (ej. cómo TucanGo usa ramas y PRs).
[MINUTO 15-20]———> IMPACTO PROFESIONAL: ¿Por qué esto define el empleo de un Ingeniero de Sistemas?
```

💡 Preguntas Trampa del Docente y Cómo Responderlas:

- **Docente:** "*¿Si se me corta el internet en el campo, puedo seguir usando Git?*"
  Respuesta: "*Sí, profesor. Git es un sistema distribuido. Todo el repositorio y su historial viven completos en la carpeta local* `.git/`*. Puedo crear ramas, hacer commits y consultar logs sin conexión; el internet solo se requiere para sincronizar con GitHub mediante* `git push` *o* `git pull`*.*"
- **Docente:** "*¿Qué diferencia hay entre* `git add .` *y* `git commit`*?*"
  Respuesta: "`git add .` *coloca los archivos modificados en la mesa de preparación (Staging Area), mientras que* `git commit` *toma la fotografía permanente de esa mesa y la guarda con un hash inmutable en la base de datos de Git.*"

---

8. Enlaces y Recursos Curados (Videos, Libros y Simuladores Interactivos)

📖 Documentación Oficial y Libros Gratuitos (La Base Académica)

- **Libro Oficial Pro Git (Español Completo)**
  El libro de referencia obligatorio de la industria, escrito por Scott Chacon y Ben Straub. Disponible gratis en formato web y PDF.
- **Documentación Oficial de GitHub (Get Started - Español)**
  Guías oficiales paso a paso para configurar cuentas, llaves SSH y repositorios.

🎮 Simuladores y Laboratorios Interactivos (Para Aprender Jugando)

- **Learn Git Branching (Simulador Visual en Español)**
  La mejor herramienta interactiva del mundo para entender ramas, merges, rebases y navegación mediante un árbol visual animado.
- **GitHub Skills**
  Cursos prácticos interactivos creados por el equipo de GitHub dentro de repositorios reales.

📹 Cursos y Videos de Alta Calidad en YouTube (Español)

- **Curso de Git y GitHub desde Cero — Midudev (YouTube)**
  Explicación moderna, práctica, visual y muy entretenida de todo el ciclo de Git y GitHub.
- **Git y GitHub desde Cero para Principiantes — MoureDev (YouTube)**
  Guía paso a paso desde la instalación, configuración de terminal, comandos clave hasta la publicación en la nube.
- **Curso de Git y GitHub — Fazt Code (YouTube)**
  Explicación técnica detallada con terminal y ejemplos de trabajo en equipo.
- **Aprende Git en 15 Minutos — Traversy Media / Traducción**
  Videos síntesis ideales para un repaso veloz antes de entrar a la exposición.

Documento compilado y estructurado bajo estándares de Ingeniería de Software — Universidad de la Amazonia.